JAIN
DEEMED-TO-BE UNIVERSITY

FACULTY OF
ENGINEERING
AND TECHNOLOGY

Revolutionizing B.Tech

JAIN
DEEMED-TO-BE UNIVERSITY

Course Name: Object Oriented Programming using Java

Total Hours : 45

JGI JAIN DEEMED-TO-BE UNIVERSITY

Module 3: Inheritance and Polymorphism

Course Name: Object Oriented Programming [25ESCS03]

Total Hours : 6 + 6

JGI JAIN DEEMED-TO-BE UNIVERSITY

Table of Content

- Aim – Understanding the purpose and efficacy of object oriented programming
- Introduction
- Objective – unit wise split into 3 to 5
- Sections/Topics
- Self Assessments – 3 per section
- Activities – 1 per section
- Did You Know*
- Summary
- Terminal Questions
- Reference Links*
- Q&A*
- Thank You

*not mandatory

JGI JAIN DEEMED-TO-BE UNIVERSITY

Aim



To equip students in the fundamentals and understanding of object oriented paradigm and its efficacy in programming large projects

JGI JAIN DEEMED-TO-BE UNIVERSITY



Objective

a. Explain the fundamentals of software engineering to understand the need of object oriented programming

b. Understanding the java language fundamentals to implement object oriented programming

c. Understanding the components of Java language and its various keywords to create an object oriented paradigm

JGI JAIN DEEMED-TO-BE UNIVERSITY

Course Outcome (Cos)

On completion of the course the student will be able to:

CO1: Understand Java syntax, object-oriented principles, and control structures.
CO2: Implement object-oriented concepts using classes, objects, and constructors.
CO3: Apply inheritance, polymorphism, and dynamic method dispatch in Java.
CO4: Design modular Java applications using interfaces, abstract classes, and packages.
CO5: Handle exceptions, file I/O, and apply collections in Java applications.
CO6: Analyze and implement advanced Java programming features such as multi-threading, interfaces, wrapper classes, and package structuring for efficient, modular, and concurrent application development.

JAIN
DEEMED-TO-BE UNIVERSITY

Module 3

Inheritance and Polymorphism – Inheritance - Single, multilevel, hierarchical inheritance, Syntax and method reuse, Method Overriding - Redefining methods, Access modifiers and annotations, Polymorphism - Compile-time vs Runtime, Method overloading vs overriding, super, final - Accessing parent class constructors/methods, Restricting inheritance/overriding.

JGI JAIN DEEMED-TO-BE UNIVERSITY

Module 2

Lab Practice: (6 Hours)

1. Lab exercise 1: Implement single and multilevel inheritance (Employee) (Inheritance hierarchy)
2. Lab exercise 2: Demonstrate method overriding in subclass (Runtime polymorphism)
3. Lab exercise 3: Use super to access parent class members (Keyword usage)
4. Lab exercise 4: Write a program that uses dynamic method dispatch (Late binding and reference polymorphism)

JGI JAIN DEEMED-TO-BE UNIVERSITY

What is Inheritance?

- **Definition:**
  - Inheritance is a mechanism where one class acquires the properties (variables) and behaviors (methods) of another class.
  - The "IS-A" Relationship: Inheritance represents an "IS-A" relationship.
    - Example: A Car is a Vehicle. A Surgeon is a Doctor.
  - **Key Terminology:**
    - Super Class (Parent/Base Class): The class whose features are inherited.
    - Sub Class (Child/Derived Class): The class that inherits the other class.

JGI JAIN DEEMED-TO-BE UNIVERSITY

Why Use Inheritance?

1. Code Reusability (Method Reuse):

This is the primary reason for inheritance.You can reuse fields and methods of the existing class without having to rewrite the same code in the new class.

2. Extensibility:

You can add new features to a child class without modifying the parent class.

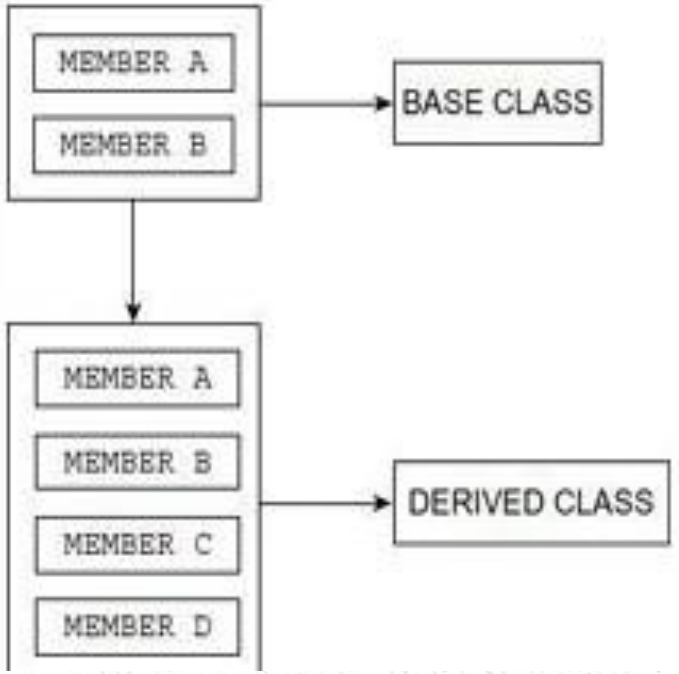
3. Method Overriding:

Inheritance is strictly required to achieve Runtime Polymorphism (Method Overriding).

JGI JAIN DEEMED-TO-BE UNIVERSITY

Inheritance

```
class Parent
{
    // properties and methods
}
class Child extends Parent
{
    // additional properties and methods
}
```



JGI JAIN DEEMED-TO-BE UNIVERSITY

Single Inheritance

```
class Employee
{
   float salary = 40000;
}

class Programmer extends Employee
{
   int bonus = 10000;
}

public class MainClass
{
   public static void main(String[] args)
   {
      Programmer p = new Programmer();
      System.out.println("Programmer salary is: " + p.salary);
      System.out.println("Bonus of Programmer is: " + p.bonus);
   }
}
```

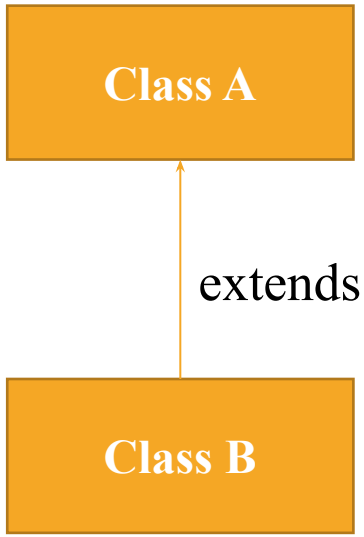
JGI JAIN DEEMED-TO-BE UNIVERSITY

Types of inheritance

There are five types of inheritance.

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

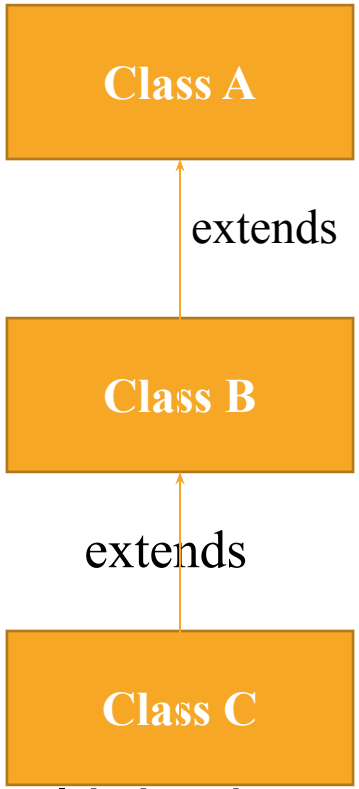
Java Single Inheritance



In single inheritance, a single subclass extends from a single superclass.

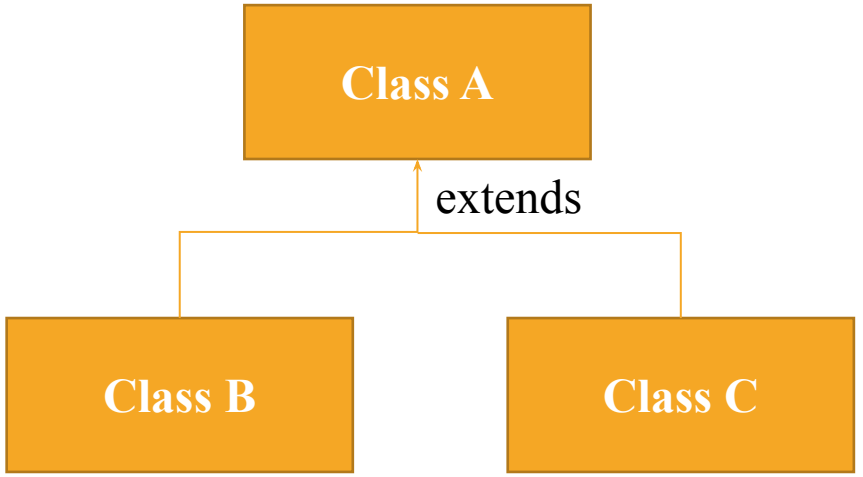
JGI JAIN DEEMED-TO-BE UNIVERSITY

Multilevel Inheritance



In multilevel inheritance, a subclass extends from a superclass and then the same subclass acts as a superclass for another class

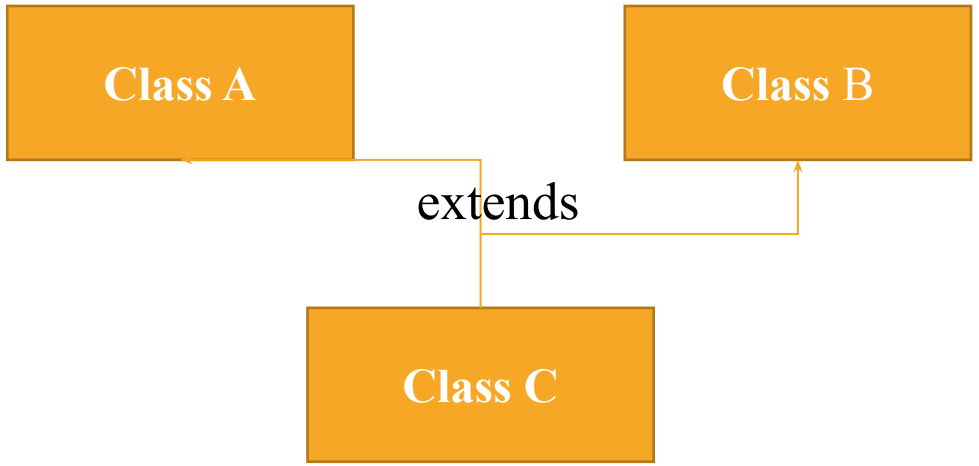
Hierachical Inheritance



In hierarchical inheritance, multiple subclasses extend from a single superclass

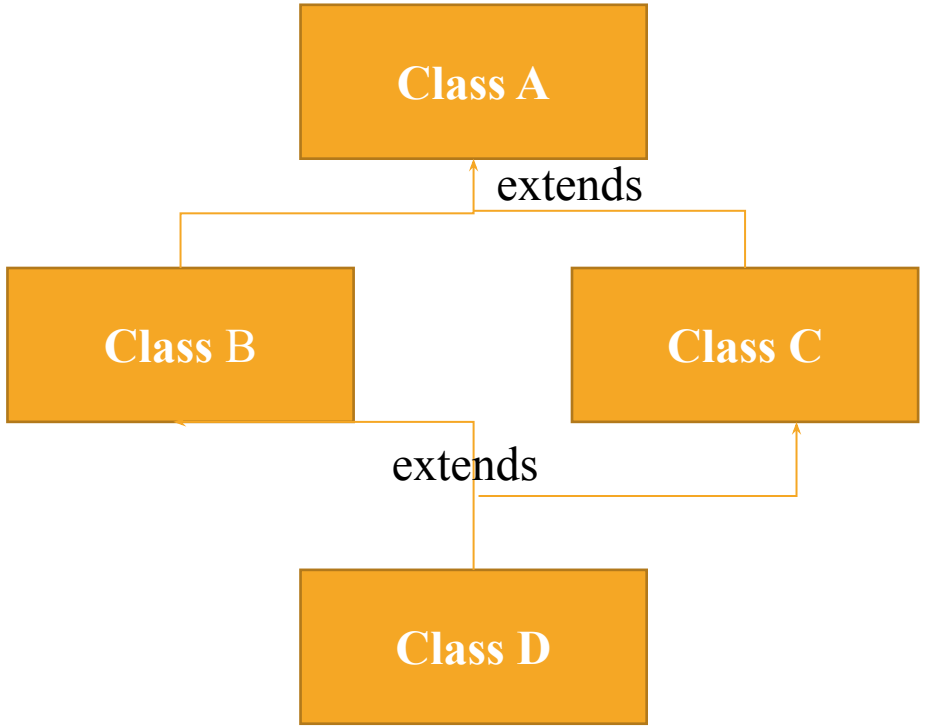
JGI JAIN DEEMED-TO-BE UNIVERSITY

Multiple Inheritance



- In multiple inheritance, a single subclass extends from multiple superclasses
- Java doesn't support multiple inheritance. However, we can achieve multiple inheritance using interfaces

Hybrid Inheritance



- Hybrid inheritance is a combination of two or more types of inheritance
- Here, we have combined hierarchical and multiple inheritance to form a hybrid inheritance

JGI JAIN DEEMED-TO-BE UNIVERSITY

Single Inheritance

```
class Animal
{
   void eat()
   {
      System.out.println("Animal is eating");
   }
}

class Dog extends Animal
{
   void bark()
   {
      System.out.println("Dog is barking");
   }
}
```

```
public class MainClass
{
   public static void main(String[] args)
   {
      Dog d = new Dog();
      d.eat();
      d.bark();
   }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

Multilevel Inheritance

```
class Animal {
   void eat()
   {
      System.out.println("Animal is eating");
   }
}
class Dog extends Animal
{
   void bark()
   {
      System.out.println("Dog is barking");
   }
}
class Puppy extends Dog
{
   void weep()
   {
      System.out.println("Puppy is weeping");
   }
}
```

```
public class MainClass
{
   public static void main(String[] args)
   {
      Puppy p = new Puppy();
      p.eat();
      p.bark();
      p.weep();
   }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

Hierarchical Inheritance

```
class Animal {
   void eat()
   {
      System.out.println("Animal is eating");
   }
}
class Dog extends Animal
{
   void bark()
   {
      System.out.println("Dog is barking");
   }
}
class Cat extends Animal
{
   void meow()
   {
      System.out.println("Cat is meowing");
   }
}
```

```
public class MainClass
{
   public static void main(String[] args)
   {
      Dog d = new Dog();
      Cat c = new Cat();

      d.eat();
      d.bark();

      c.eat();
      c.meow();
   }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

Method Reuse

Concept:

- A subclass automatically inherits the non-private methods of its superclass.

Benefit:

- You write the method once in the parent class and call it using the child class object, eliminating duplicate code.

JGI JAIN DEEMED-TO-BE UNIVERSITY

Method Overriding (Runtime Polymorphism)

- Method Overriding is when a child class provides its own specific implementation of a method that is already defined in its parent class.
- **Rules for Method Overriding:**
  - The method name must be the same.
  - The parameter list must be the same.
  - There must be an IS-A relationship (inheritance).The return type must be the same or a subtype.

JGI JAIN DEEMED-TO-BE UNIVERSITY

Method Overriding (Runtime Polymorphism)

```
class Animal
{
   void sound()
   {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal
{
   @Override
   void sound()
   {
      System.out.println("Dog barks");
   }
}
```

```
public class MainClass
{
   public static void main(String[] args)
   {
      Dog d = new Dog();
      d.sound();

      Animal a = new Animal();
      a.sound();

   }
}
```

Output:

Dog barks
Animal makes a sound

JGI JAIN DEEMED-TO-BE UNIVERSITY

Method Overriding (Runtime Polymorphism)

- **What is an Annotation?**
  - An annotation is a special label/tag attached to a method, class, or variable that gives extra information to the compiler.
- **@Override Annotation:**
  - Tells the compiler that this method is intentionally overriding a parent class method.
  - If the method signature does NOT match the parent, the compiler throws an error immediately.
  - Prevents accidental mistakes like typos in method names.
- **Best Practice:** Always use @Override when overriding methods.

JGI JAIN DEEMED-TO-BE UNIVERSITY

Access Modifiers

- Access modifiers determine the visibility of variables and methods from the parent class to the child class.
  - **public:** Accessible everywhere. Inherited by subclasses.
  - **protected:** Accessible within the same package, and in subclasses (even in different packages). Ideal for inheritance.
  - **default (no modifier):** Accessible only within the same package. Inherited only if the subclass is in the same package.
  - **private:** Accessible only within the class it is declared. Never inherited by subclasses.

JGI JAIN DEEMED-TO-BE UNIVERSITY

Access Modifiers

- Access Modifiers control the visibility and accessibility of classes, methods, and variables.

Modifier	Same Class	Same Package	Subclass	Other Package
private	✅	❌	❌	❌
default	✅	✅	❌	❌
protected	✅	✅	✅	❌
public	✅	✅	✅	✅

JGI JAIN DEEMED-TO-BE UNIVERSITY

Access Modifiers

```
class Employee {

    public String name = "John";
            // accessible everywhere
    private String company = "TechCorp";
            // only inside Employee
    protected double salary = 50000;
            // accessible in subclass
    public String getCompany() {
        return company;
    }
}
class Programmer extends Employee {

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Company: " +
getCompany());
        System.out.println("Salary: " + salary);
    }
}
```

```
public class MainClass {

    public static void main(String[] args) {

        Programmer p = new Programmer();
        p.display();
    }
}
```

Output:

```
Name: John
Company: TechCorp
Salary: 50000.0
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

Polymorphism

JGI JAIN DEEMED-TO-BE UNIVERSITY

Polymorphism

- **Definition:**
  - Polymorphism means "many forms". It is the ability of a single method or object to behave differently based on the context.
- **Real-life Analogy:**
  - A person behaves differently as a Son, Employee, and Friend — same person, different behavior.
  - A Dog sounds differently from a Cat, but both are Animals.
- **Two Types of Polymorphism in Java:**
  - Compile-time Polymorphism (Static) → Method Overloading
  - Runtime Polymorphism (Dynamic) → Method Overriding

JGI JAIN DEEMED-TO-BE UNIVERSITY

1. Compile-time Polymorphism

- **Definition:**
  - When the method to be called is determined at compile time by the Java compiler, it is called Compile-time Polymorphism.
- **Also Known As:** Static Polymorphism / Early BindingAchieved By: Method Overloading
- **How it works:** The compiler looks at the method name + parameters and decides which method to call before the program runs.
- **Key Point:** Same method name, but different number or type of parameters in the same class.

JGI JAIN DEEMED-TO-BE UNIVERSITY

1. Compile-time Polymorphism

```
class Calculator
{
   int add(int a, int b)
   {
      return a + b;
   }
   int add(int a, int b, int c)
   {
      return a + b + c;
   }
   double add(double a, double b)
   {
      return a + b;
   }
}
```

```
public class MainClass
{
   public static void main(String[] args)
   {
      Calculator calc = new Calculator();
      System.out.println("Sum (2 ints): " + calc.add(10, 20));
      System.out.println("Sum (3 ints): " + calc.add(10, 20,
30));
      System.out.println("Sum (doubles): " + calc.add(1.5,
2.5));
   }
}
```

Output:

```
Sum (2 ints): 30
Sum (3 ints): 60
Sum (doubles): 4.0
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

2. Runtime Polymorphism

- **Definition:**
  - When the method to be called is determined at runtime based on the actual object, it is called Runtime Polymorphism.
- **Also Known As:** Dynamic Polymorphism / Late Binding / Dynamic Method Dispatch
- **Achieved By:** Method Overriding
- **How it works:** A parent class reference variable holds a child class object. The JVM decides at runtime which overridden method to execute.

JGI JAIN DEEMED-TO-BE UNIVERSITY

2. Runtime Polymorphism

```
class Animal {
    void sound() {
        System.out.println("Animal makes a
sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
public class MainClass {
    public static void main(String[] args) {
        Animal a1 = new Animal();
        a1.sound();

        Animal a2 = new Dog(); // Parent ref, Child object
        a2.sound();          // Dog's method called at
runtime
    }
}
```

Output:

```
Animal makes a sound
Dog barks
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

Compile-time vs Runtime Polymorphism

Feature	Compile-time Polymorphism	Runtime Polymorphism
Also Known As	Static Binding / Early Binding	Dynamic Binding / Late Binding
Mechanism	Method Overloading	Method Overriding
When is it resolved?	During Compilation (by the Compiler)	During Execution (by the JVM)
Flexibility	Less flexible, execution is faster.	Highly flexible, execution is slightly slower.
Inheritance Required?	No	Yes (Strictly requires IS-A relationship)

JGI JAIN DEEMED-TO-BE UNIVERSITY

Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Location	Happens within the **same class**.	Happens between **two classes** (Parent & Child).
Method Name	Must be the SAME.	Must be the SAME.
Parameters	Must be **DIFFERENT** (number or type).	Must be exactly the **SAME**.
Return Type	Can be same or different.	Must be the **SAME** (or covariant type).
Use Case	To increase readability of the program.	To provide a specific implementation of a parent's method.

JGI JAIN DEEMED-TO-BE UNIVERSITY

The super Keyword

- The super keyword in Java is a reference variable used to refer to the immediate parent class object.
- **Primary Uses of super:**
  1. **Access Parent Variables:** Used to access parent class variables if the child class has a variable with the exact same name (resolves variable hiding).
  2. **Invoke Parent Methods:** Used to invoke parent class methods if the child class has overridden them.
  3. **Invoke Parent Constructors:** super() is used to call the parent class's constructor from the child class's constructor.

JGI JAIN DEEMED-TO-BE UNIVERSITY

super - Accessing Variables and Methods

- The child class overrides the parent's method and hides the parent's variable. We use super to reach back to the parent's version.

JGI JAIN DEEMED-TO-BE UNIVERSITY

super - Accessing Variables and Methods

```
class Animal {
    String type = "Animal";
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    String type = "Dog";
    @Override
    void sound()
    {
        System.out.println("Dog barks");
        super.sound();        // calls Animal's sound()
    }
    void display()
    {
        System.out.println("Parent type: " + super.type); // Animal
        System.out.println("Child type: " + this.type);   // Dog
    }
}
```

```
public class MainClass
{
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
        d.display();
    }
}
```

Output:

```
Dog barks
Animal makes a sound
Parent type: Animal
Child type: Dog
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

super() - Accessing Parent Constructors

```
class Person
{
   Person()
   {
      System.out.println("Person (Parent) Constructor created");
   }
}

class Student extends Person
{
   Student()
   {
      super(); // Invokes immediate parent class constructor
      System.out.println("Student (Child) Constructor created");
   }
}
```

```
public class SuperConstructorDemo
{
   public static void main(String args[])
   {
      Student s = new Student();
   }
}
```

Output:

```
Person (Parent) Constructor created
Student (Child) Constructor created
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

The final Keyword

Definition:

- The final keyword is used to restrict the user from modifying something.

Usage	Meaning
`final variable`	Value cannot be changed (constant)
`final method`	Method cannot be overridden by child class
`final class`	Class cannot be inherited (no subclass allowed)

- Real-life Analogy:
  - A final variable is like a fixed rule that cannot be changed.
  - A final method is like a sealed process no one can modify.
  - A final class is like a locked door — no one can extend it.

JGI JAIN DEEMED-TO-BE UNIVERSITY

final Variable & Method – Program

```
class Vehicle
{
   final int speed = 120;      // constant, cannot change
   final void fuelType()
   {
      // cannot be overridden
      System.out.println("Runs on Petrol");
   }
}
class Car extends Vehicle
{
   void display()
   {
      // speed = 150;  ERROR! Cannot assign to final variable
      System.out.println("Max Speed: " + speed);
      fuelType();
   }
}
```

```
public class MainClass
{
   public static void main(String[] args)
   {
      Car c = new Car();
      c.display();
   }
}
```

Output:

Max Speed: 120
Runs on Petrol

JGI JAIN DEEMED-TO-BE UNIVERSITY

final Variable & Method – Program

```
class Vehicle {
    final int speed = 120;         // constant, cannot change

    final void fuelType() {        // cannot be overridden
        System.out.println("Runs on Petrol");
    }
}
class Car extends Vehicle {
    void display() {
        // speed = 150;  ERROR! Cannot assign to final variabl
        System.out.println("Max Speed: " + speed);
        fuelType();
    }
    void fuelType()
    {       // cannot be overridden
        System.out.println("Runs on Petrol 2222");
    }
}
```

STDIN

Input for the program (Optional)

Output:

```
MainClass.java:20: error: fuelType() in
    void fuelType()
         ^
  overridden method is final
1 error
```

JGI JAIN DEEMED-TO-BE UNIVERSITY

final Class – Restricting Inheritance – Program

```
final class Vehicle
{
   void display()
   {
      System.out.println("This is a Vehicle");
   }
}
// class Car extends Vehicle { }
// ERROR! Cannot inherit from a final class

public class MainClass
{
   public static void main(String[] args)
   {
      Vehicle v = new Vehicle();
      v.display();
   }
}
```

Output:

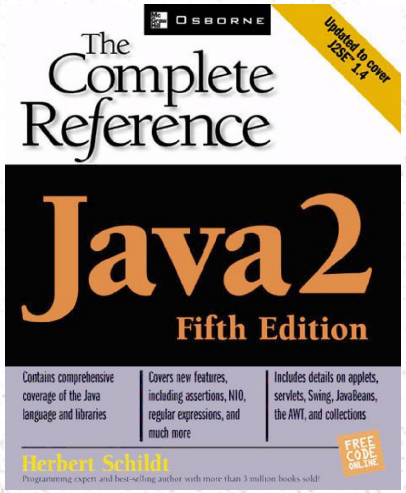
This is a Vehicle

JGI JAIN DEEMED-TO-BE UNIVERSITY

Reference Links

1. Java 2 Complete Reference 5th Edition - https://isa.atu.edu.iq/wp-content/uploads/2020/02/The-Complete-Reference-java-5th.pdf

2. Java 2 Complete Reference 7th Edition https://drive.google.com/file/d/1Y4jzNIUJ_3GhsiMpAm5lET09rhbOwizr/view?usp=sharing



Thank you

JGI JAIN
DEEMED-TO-BE UNIVERSITY